Java Programming
Creating a stand-alone Java application using JavaFX

Overview

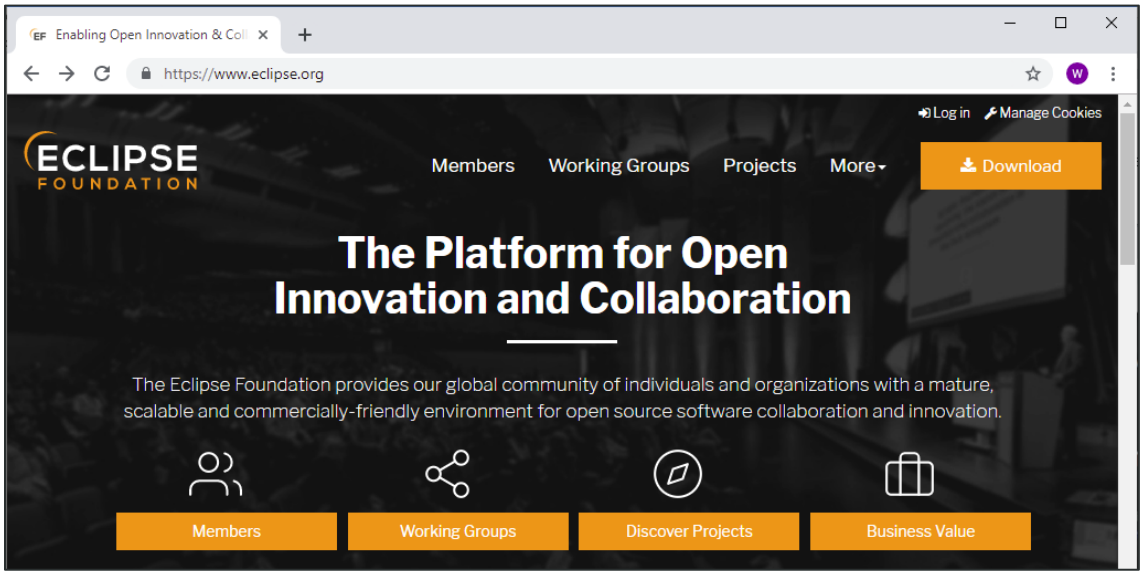
In this tutorial you will learn how to create a Java FX application in eclipse. The application will be a simple voting application that you can create a JAR file of so that it can be ran outside of the development environment.



Installing JavaFX in Eclipse

You need to get the latest version of JavaFX to install in Eclipse.

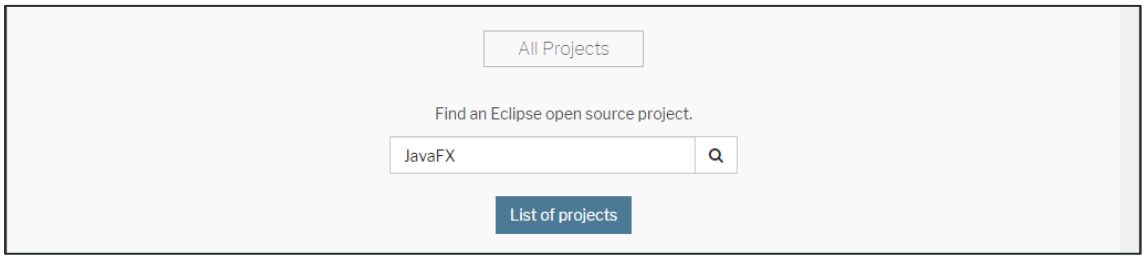
1. Go to www.eclipse.org in your browser to get the address to download the latest version of FX.
2. Click the **Download** button to access the download center.



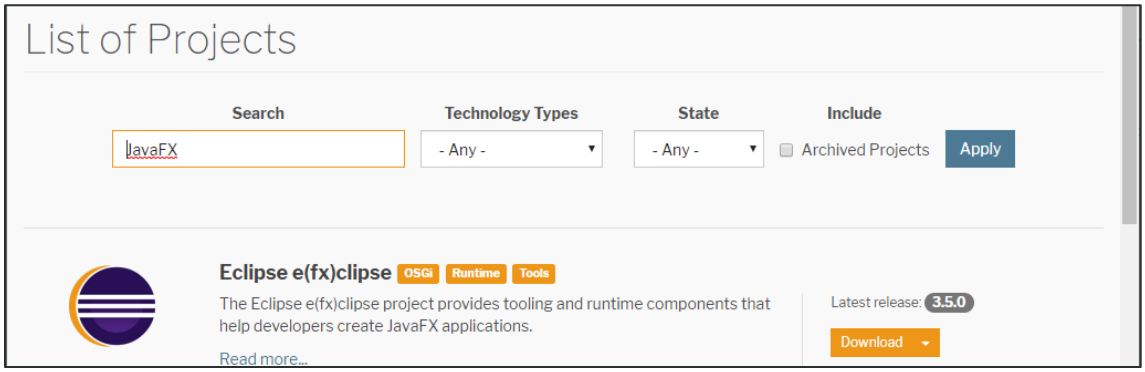
ORACLE Academy

Copyright © 2026, Oracle and/or its affiliates. Oracle®, Java, MySQL and NetSuite are registered trademarks of Oracle and/or its affiliates. Other names may be trademarks of their respective owners.

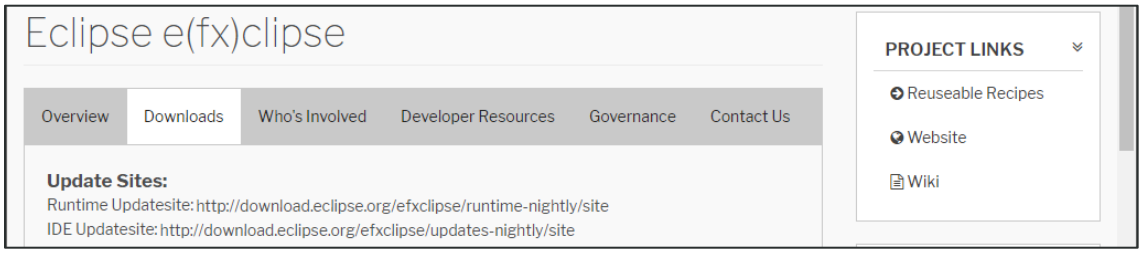
3. Scroll down to the search bar and type **JavaFX** as your search criteria before clicking the **Search** button.



4. The JavaFX project will be displayed. Click on the **Download** button.



5. The IDE update site address will allow you to easily install JavaFX in Eclipse. Select and copy the address for use in the Eclipse IDE.



6. At the time of writing the address for JavaFX was:
   http://download.eclipse.org/efxclipse/updates-nightly/site

   You can copy this address if you can't access the eclipse site.

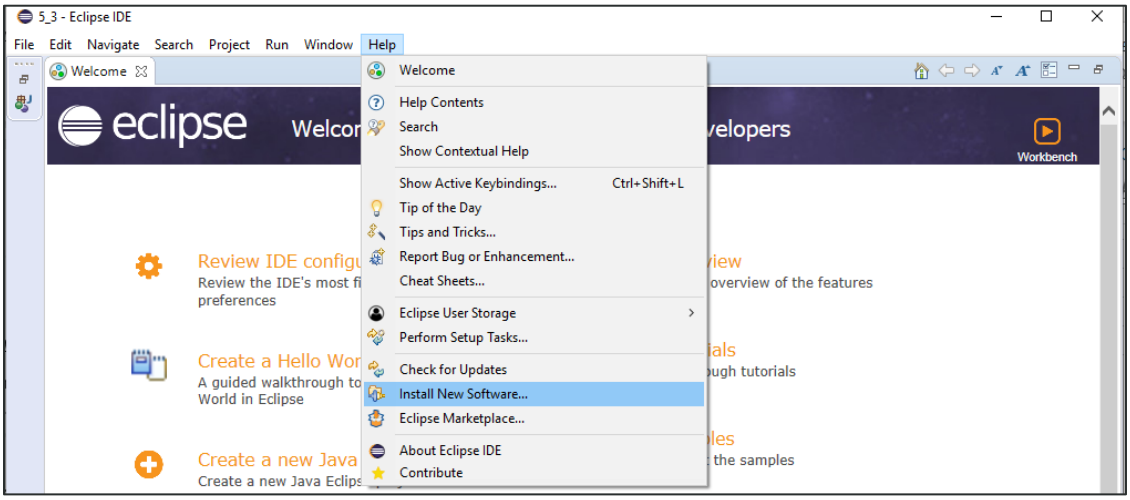
ORACLE Academy

Copyright © 2026, Oracle and/or its affiliates. Oracle®, Java, MySQL and NetSuite are registered trademarks of Oracle and/or its affiliates. Other names may be trademarks of their respective owners.

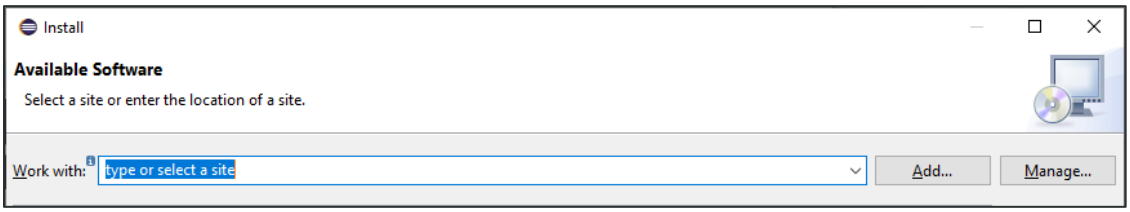
Adding JavaFX to Eclipse

FX is not installed by default in Eclipse so must be added.

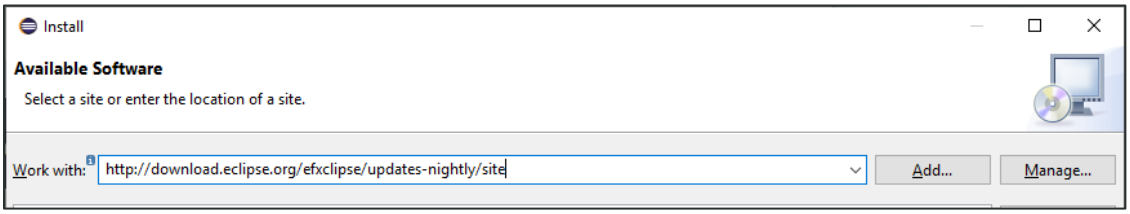
1. Start Eclipse and select **Help** from the menu.
2. Select **Install New Software** from the drop-down menu.



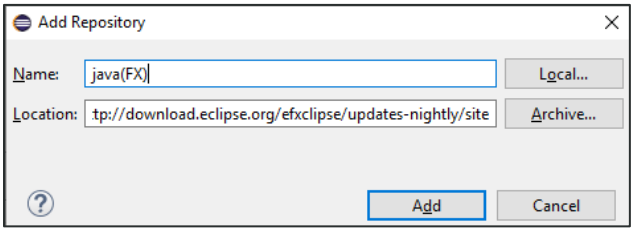
3. In the work with box that displays a "**Type or select site**" message paste the IDE Update address that you copied from the Eclipse website.



4. Click the **Add** button.



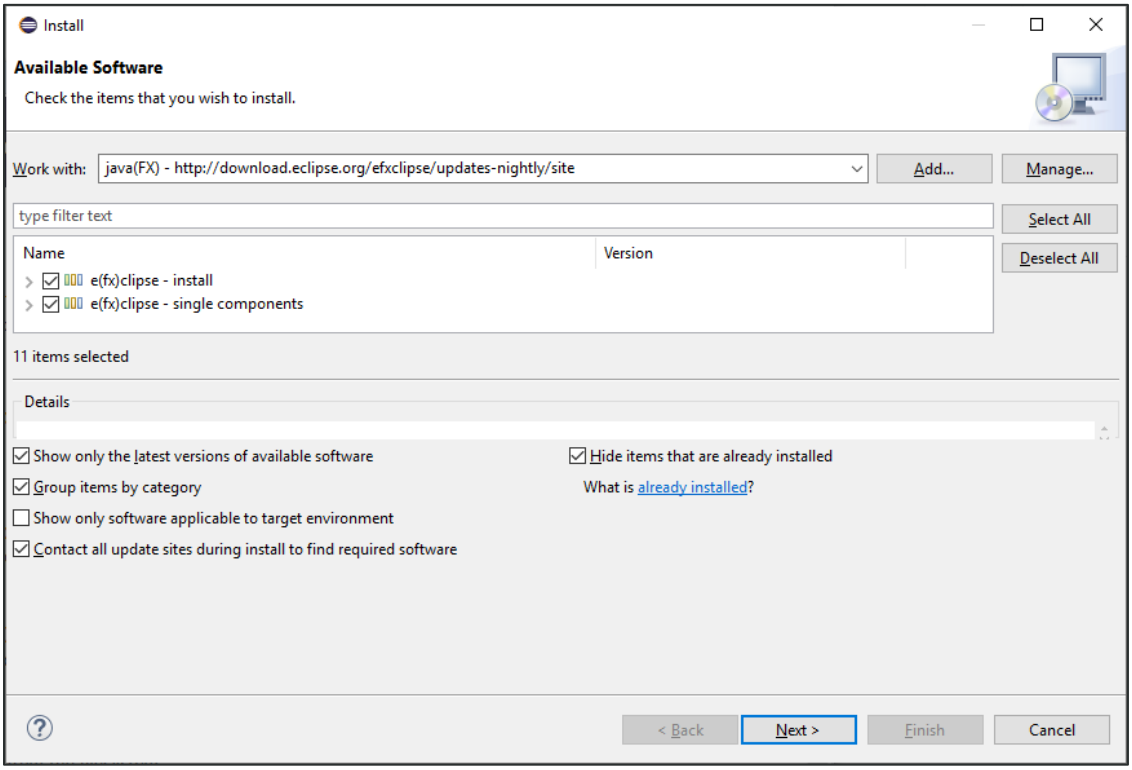
5. Name the repository as **java(FX)** and click **Add**.



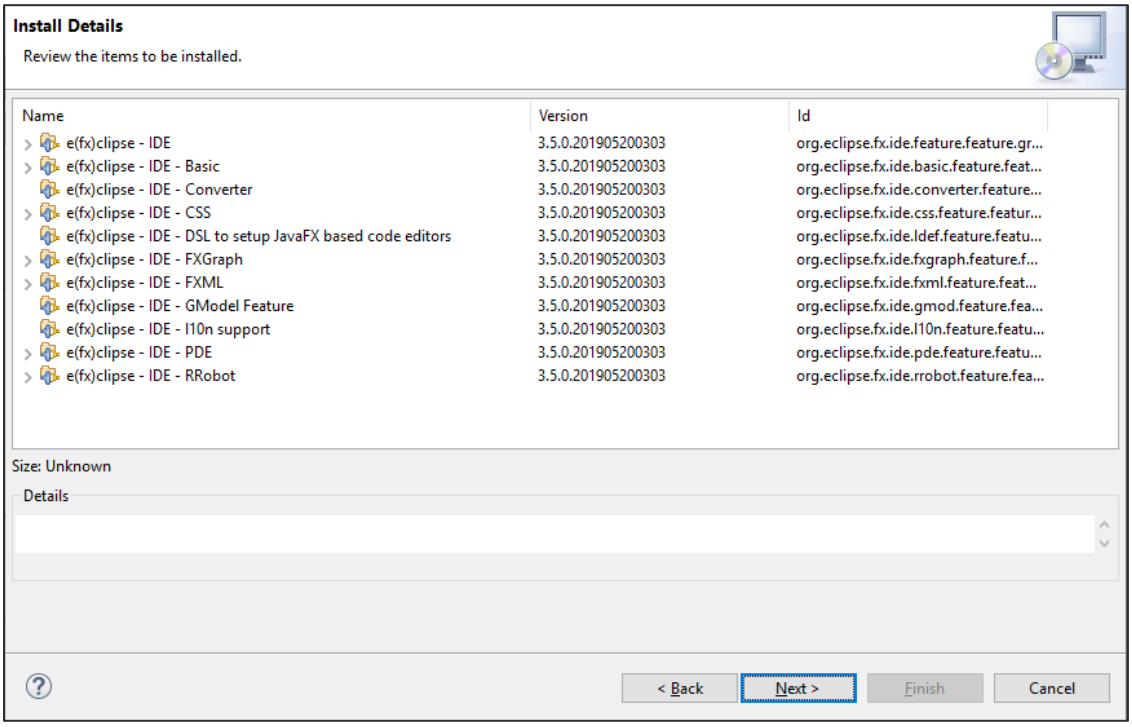
ORACLE Academy

Copyright © 2026, Oracle and/or its affiliates. Oracle®, Java, MySQL and NetSuite are registered trademarks of Oracle and/or its affiliates. Other names may be trademarks of their respective owners.

6. Tick both **e(fx)** boxes to select all of the components and click the **Next >** button.



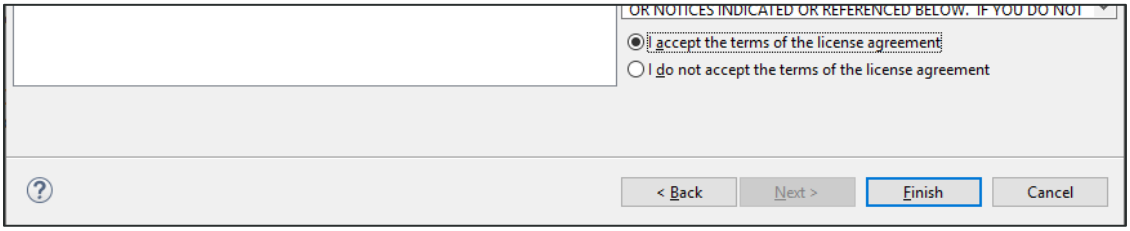
7. You will be shown a list of the items that will be installed. Click **Next >**.



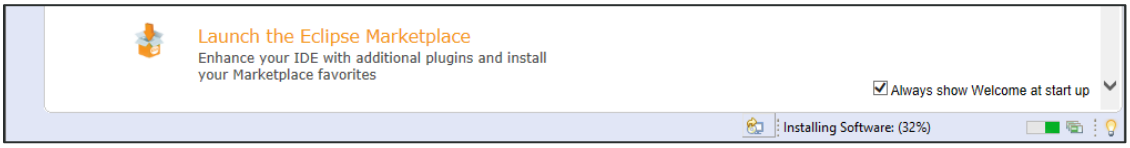
ORACLE Academy

Copyright © 2026, Oracle and/or its affiliates. Oracle®, Java, MySQL and NetSuite are registered trademarks of Oracle and/or its affiliates. Other names may be trademarks of their respective owners.

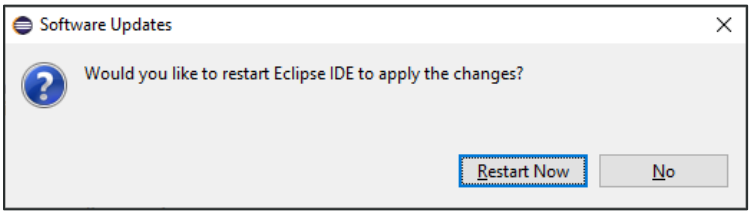
8. Accept the terms of the license agreement and click **Finish**.



9. Eclipse will now install the selected software (this may take a few minutes).



10. When the software is installed you will be asked to restart Eclipse, click **Restart Now** to complete the install.



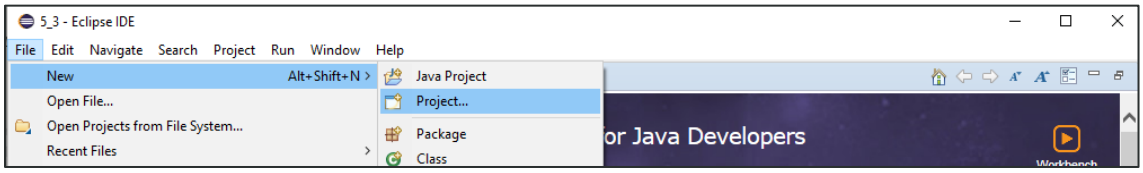
ORACLE Academy

Copyright © 2026, Oracle and/or its affiliates. Oracle®, Java, MySQL and NetSuite are registered trademarks of Oracle and/or its affiliates. Other names may be trademarks of their respective owners.

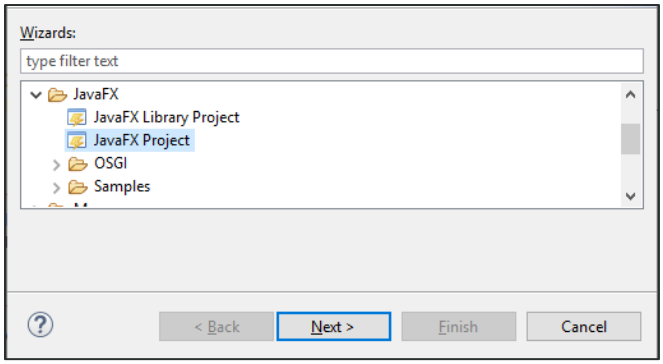
Creating a JavaFX application in Eclipse

To create a JavaFX application in Eclipse you need to create a JavaFX project.

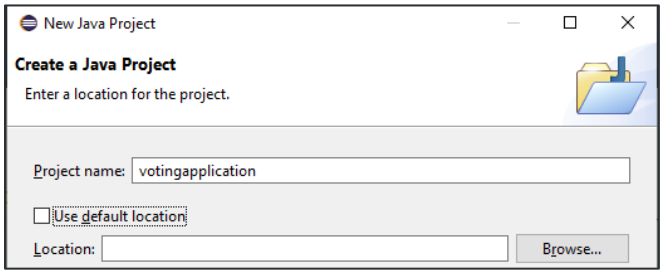
1. Select **File**, **New** and **Project** from the menu from the restarted Eclipse IDE.



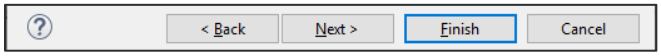
2. Select **JavaFX** and then **JavaFX Project** from the drop-down list and click **Next >**.



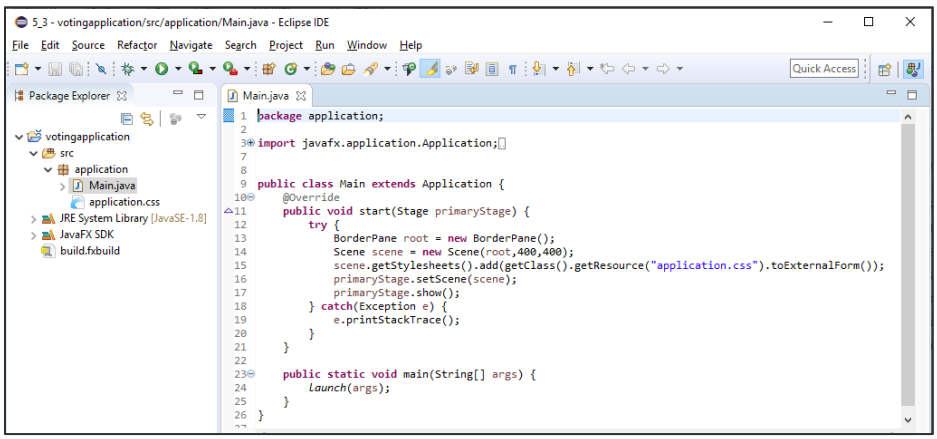
3. Call the project **votingapplication** and select a workspace location or tick the box to use the default location.



4. Leave the settings as their default and click **Finish** to create the JavaFX project.



5. Once the building workspace process is complete you can open the **Main.java** file to see the base FX code that is provided for you to build on.



ORACLE Academy

Copyright © 2026, Oracle and/or its affiliates. Oracle®, Java, MySQL and NetSuite are registered trademarks of Oracle and/or its affiliates. Other names may be trademarks of their respective owners.

Coding a JavaFX application

When you create a JavaFX project you combine components and action listeners to create an interactive GUI application. The following application will allow users to vote for either Java or SQL as their favorite topic.

1. Create the following instance fields above the `start()` method in the code. These will be used to keep the score and for the content of a label.

```
public class Main extends Application {
   int cat1Score=0;
   String cat1 = "Java Team";
```

2. Create a label that will display the category for the vote using the following code at the start of the `main` method.

```
public void start(Stage primaryStage) {
   //create and position the Java category label
   Label cat1Lbl = new Label(cat1);
   cat1Lbl.setLayoutX(110);
   cat1Lbl.setLayoutY(10);
   cat1Lbl.setTextFill(Color.RED);
```

   Note:

   - `Label cat1Lbl = new Label(cat1)` – creates a new label that displays the value of `cat1`
   - `cat1Lbl.setLayoutX(110)` – sets the `X` coordinate of the label
   - `cat1Lbl.setLayoutY(10)` – sets the `Y` coordinate of the label
   - `cat1Lbl.setTextFill(Color.RED)` – Sets the text color of the label

3. You will have to import libraries to use these components. Always make sure you import the **javaFX** library.

```
import javafx.application.Application;
import javafx.stage.Stage;
import javafx.scene.Scene;
import javafx.scene.control.Label;
import javafx.scene.layout.BorderPane;
import javafx.scene.paint.Color;
```

4. You will now create another label that will display the number of votes cast. This will be displayed under the previous label and centered to the position of cat1Lbl.

```
//create and position the category 1 scores label
Label score1Lbl  = new Label(""+cat1Score);
score1Lbl.setLayoutX(130);
score1Lbl.setLayoutY(40);
score1Lbl.setTextFill(Color.RED);
```

ORACLE Academy

Copyright © 2026, Oracle and/or its affiliates. Oracle®, Java, MySQL and NetSuite are registered trademarks of Oracle and/or its affiliates. Other names may be trademarks of their respective owners.

5. A button is needed that the user can click to cast their vote. This will be positioned under the previous two labels. You will have to import the **Button** library, remember to choose the **fx** option.

```
//Create and position the category 1 button
Button cat1Btn = new Button("Vote " + cat1);
cat1Btn.setLayoutX(90);
cat1Btn.setLayoutY(80);
```

6. To make a component interactive you need to add an event handler to the component. Use the drop-down list of content prompts in Eclipse to create the following action listener that uses a Lambda expression.

```
cat1Btn.setOnAction((ActionEvent event) -> {

}); //end lambda expression
```

You will need to import the **javaFX ActionEvent**.

7. The code inside the handle method is fired whenever the button is clicked. Add the code to increment the number of votes (stored as an instance field) and update the text in the label to show the new value.

```
cat1Btn.setOnAction((ActionEvent event) -> {
   cat1Score++;
   score1Lbl.setText(""+cat1Score);
}); //end lambda expression
```

8. Although you have created the components you have not yet added them to your application so if you were to run your code now you would not see them. To add the components you have to add them to an FX layout manager. Currently the application uses a `BorderPane` layout (you can learn more about these through the Java API), but you are going to use a `Group` layout that allows the placing of components using `X` and `Y` coordinates.

Update the existing code:

```
BorderPane root = new BorderPane();
```

to

```
Group root = new Group();
```

You will have to import the **javafx.scene.Group** library (you can delete the `BorderPane` library import as you won't be using it).

ORACLE Academy

Copyright © 2026, Oracle and/or its affiliates. Oracle®, Java, MySQL and NetSuite are registered trademarks of Oracle and/or its affiliates. Other names may be trademarks of their respective owners.

9. Add each component individually to the root node using the `getChildren().add` method:

```
Group root = new Group();
root.getChildren().add(cat1Lbl);
root.getChildren().add(score1Lbl);
root.getChildren().add(cat1Btn);
```

As you are placing them with their coordinates the order that you add them is not important (this is not always case).

10. Update the size of the scene (this is what you will add the `root` to in order to display the components in the primary stage (Window) to have a width of 300 and a height of 150.

```
Scene scene = new Scene(root, 400, 150);
```

11. You will not be using the stylesheets in this exercise so you can remove the line that starts with `scene.getStyleSheets()`.

12. Add the following code that sets a title on the window and sets the width and height of the window equal to the dimensions of the scene.

```
primaryStage.setTitle("Voting Panel!");
primaryStage.setMinWidth(scene.getWidth());
primaryStage.setMinHeight(scene.getHeight());
primaryStage.setScene(scene);
```

13. You can now run and test that your code works.

14. Use the following information to create a second set of labels and buttons:

    a. An integer instance field named `cat2Score` that is initialized to 0.

    b. A `String` instance filed named `cat2` that is initialized to "**SQL Team**"

    c. Create a `cat2Lbl` at position 230, 10 that displays the text in blue.

    d. Create a `score2Lbl` at position 250, 40 that displays the score in blue.

    e. Create a button named `cat2Btn` that displays `vote` plus the value of `cat2`. Its position should be 210, 80.

    f. Add an event handler for the button that increases `cat2Score` by one and updates the text value.

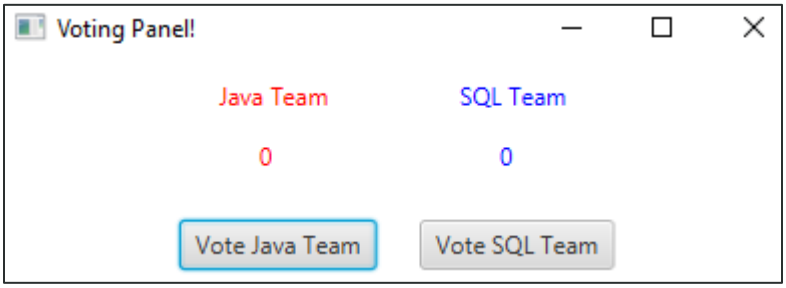
    g. Add the components to the `Group` layout manager.

    h. Run and test that your code works.

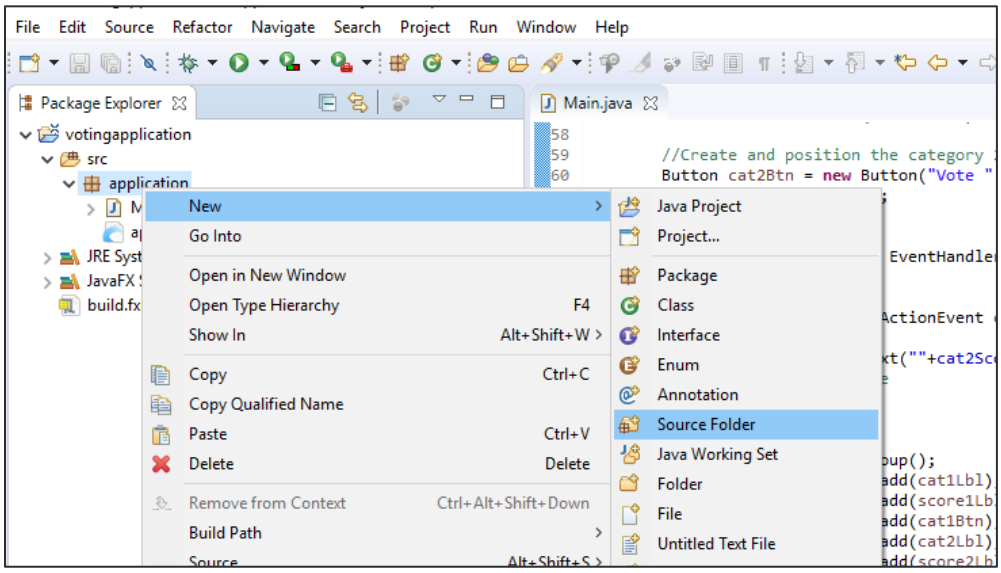
ORACLE Academy

Copyright © 2026, Oracle and/or its affiliates. Oracle®, Java, MySQL and NetSuite are registered trademarks of Oracle and/or its affiliates. Other names may be trademarks of their respective owners.

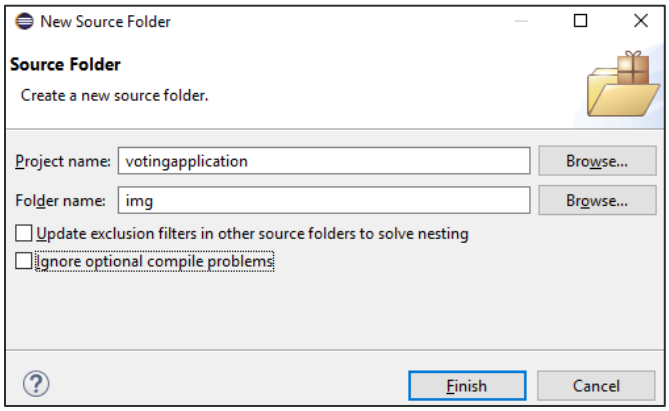
i. Your application should look like this:



15. You will now add images to the application by including them in a resources folder with the application. In Eclipse, right-click on the application package and select **New** > **Source Folder**.



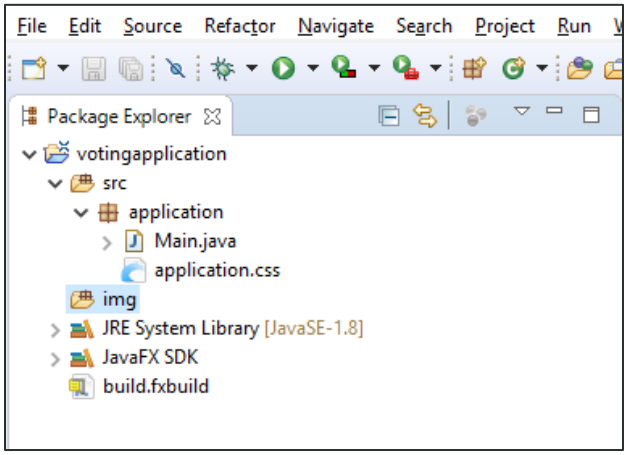
16. Name the source folder **img** and then click **Finish**.



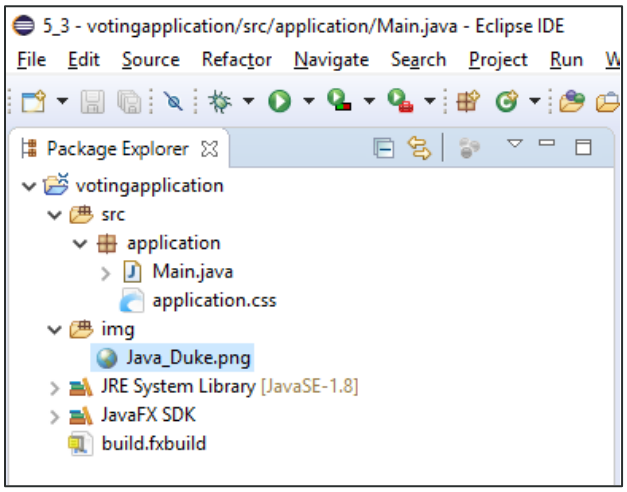
ORACLE Academy

Copyright © 2026, Oracle and/or its affiliates. Oracle®, Java, MySQL and NetSuite are registered trademarks of Oracle and/or its affiliates. Other names may be trademarks of their respective owners.

17. The **img** folder is displayed in your project directory.



18. Drag the image file **Java_Duke.png** from your local directory into the **img** folder. Choose **copy link** and then **OK** to complete the process.



19. Above the `try catch` block in your code add the following line of code that creates a reference (`jImage`) to the Java_Duke image in the **img** folder. You will need to import the **fx** image library.

```
//create a link to the Java image
Image jImage =
new Image(Main.class.getResourceAsStream("/Java_Duke.png"));
```

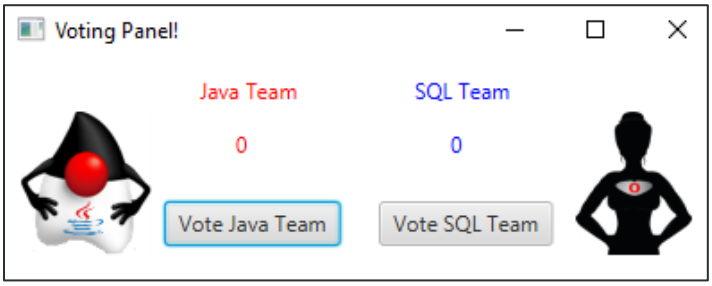
20. To display the image in the application you have to place it in an `ImageView` object that you can then use to set the display coordinates. Add the following code.

```
//Setting the image view and place the image
ImageView jImgView = new ImageView(jImage);
jImgView.setX(10);
jImgView.setY(30);
```

ORACLE Academy

Copyright © 2026, Oracle and/or its affiliates. Oracle®, Java, MySQL and NetSuite are registered trademarks of Oracle and/or its affiliates. Other names may be trademarks of their respective owners.

21. Use the following information to add an image for the SQL Team vote.
    a. Drag the **Datagirl.png** image file into the **img** source folder.
    b. Create a reference to the **Datagirl** image named **sqlImage**.
    c. Create a **sqlImgView** using the **sqlImage** reference.
    d. Place the **sql** image at the following coordinates: X- 320, Y-30.
    e. Add the image view to the **Group** layout **manager**.
    f. Your application should look like this:



22. You now have a working app that allows votes to be cast, however the user experience can be improved with a few tweaks.

    The vote count should not be displayed on screen until the vote is over.

    a. Under the `setTextFill()` method for each score label add the following code so that the labels are not displayed to screen.

```
score1Lbl.setTextFill(Color.RED);
score1Lbl.setVisible(false);
score2Lbl.setTextFill(Color.BLUE);
score2Lbl.setVisible(false);
```

    b. Create a `private` method named `showVotes` that accepts no parameters and returns no values. This method will set the value of the scores as the text in the score labels and then display the labels to screen. Create the method above the `main` method at the bottom of the code.

```
private void showVotes() {
    score1Lbl.setText(""+cat1Score);
    score2Lbl.setText(""+cat2Score);
    score1Lbl.setVisible(true);
    score2Lbl.setVisible(true);
}//end method showVotes
```

ORACLE Academy

Copyright © 2026, Oracle and/or its affiliates. Oracle®, Java, MySQL and NetSuite are registered trademarks of Oracle and/or its affiliates. Other names may be trademarks of their respective owners.

c. You get errors as the compiler cannot recognize the `scoreLbl` components. This is because you created them locally in the `start` method. To fix this you can create the reference to the component as an instance field and then create the component object locally.

Add the following instance fields to the top of your class:

```
public class Main extends Application {
    int cat1Score=0, cat2Score = 0;
    String cat1 = "Java Team", cat2 = "SQL Team";
    private Label score1Lbl, score2Lbl;
```

d. Update the code in the **start** menu to instantiate the object referenced as an instance field instead of creating a second one. For the score labels update the code by not declaring a type at the beginning of the statement so that it does this:

```
//create and position the category 1 scores label
score1Lbl = new Label(""+cat1Score);

//create and position the category 2 scores label
score2Lbl = new Label(""+cat1Score);
```

Any components/objects that are required to be used in multiple methods should have a reference created as an instance field.

The errors that were attached to the `showVotes()` method should now have been resolved.

e. Comment out the lines in the event handlers that display the values of the scores to the labels.

```
cat1Btn.setOnAction((ActionEvent event) -> {
    cat1Score++;
    //score1Lbl.setText(""+cat1Score);
});//end lambda expression
```

Do the same for the `cat2Btn` event handler.

23. To view the votes cast,

a. Create a new button named **showBtn** that will display the text "**Show Votes**". Display the button at the following coordinates: `X`-90, `Y`-120 and add it to the `Group` layout manager.

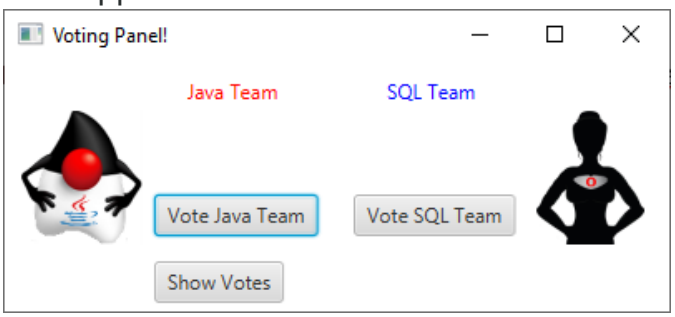
b. Call the `showVotes()` method from its event handler.

c. Run and test your program.

ORACLE Academy

Copyright © 2026, Oracle and/or its affiliates. Oracle®, Java, MySQL and NetSuite are registered trademarks of Oracle and/or its affiliates. Other names may be trademarks of their respective owners.

d. Your application should look like this:



24. You can see that the size of the buttons is different and doesn't look quite right. You can fix this by setting a preferred width for the buttons.

    a. Create an unchangeable value at the beginning of the start method:

```
public void start(Stage primaryStage) {
    //unchangeable value for the width of the buttons
final double BTN_WIDTH = 100;
```

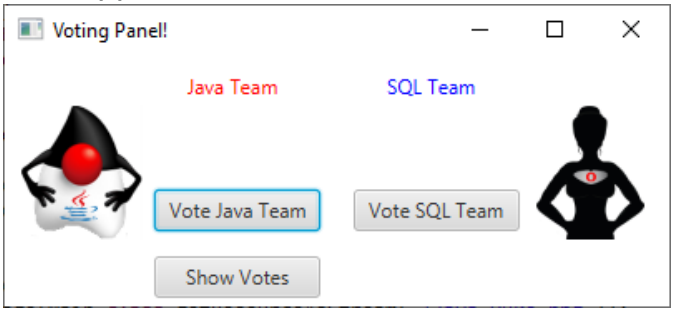
    b. In the properties list for all of the buttons use the `setPrefWidth()` method to set the width of the button equal to the `BTN_WIDTH` value.

```
showBtn.setLayoutX(90);
showBtn.setLayoutY(120);
showBtn.setPrefWidth(BTN_WIDTH);
```

25. Do the same for the `cat1Btn` and the `cat2Btn` buttons.

    a. Run your program

    b. Your application should look like this:



26. You will now add functionality that will reset the votes to zero so that voting can begin again.

    a. Create a private method that will set the values of the scores to zero and hide the scores labels.

```
private void resetVotes() {
    cat1Score=0;
    cat2Score=0;
    score1Lbl.setVisible(false);
    score2Lbl.setVisible(false);
}//end method resetVotes
```

ORACLE Academy

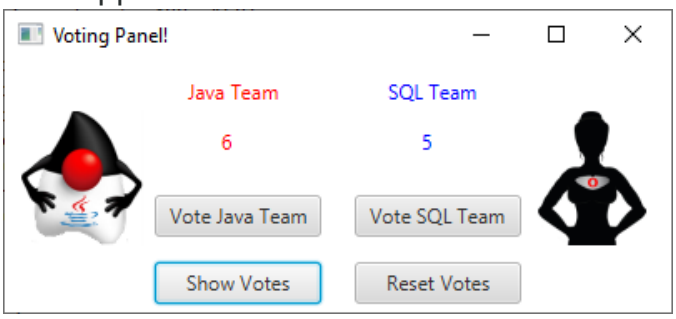
Copyright © 2026, Oracle and/or its affiliates. Oracle®, Java, MySQL and NetSuite are registered trademarks of Oracle and/or its affiliates. Other names may be trademarks of their respective owners.

b. Create a button named **resetBtn** that displays “Reset Votes” as its text, it’s positioned at 210, 120 and uses the preferred button width.

c. Call the `resetVotes()` method from its event handler.

d. Run and test your program

e. Your application should look like this:



27. There are a few issues with the application:

    a. After showing the vote values you can click the **vote** buttons to add to the totals.

    b. You can reset the votes at any time.

    c. The votes are not reset after they have been viewed.

28. Create a `private void` method named `setBtnUse` that accepts 4 `boolean` parameters that will be used to control the disabled state of the buttons. You will have to create button references as instance fields for this to work (remember to remove the **Button** type from the **start** method)

```
private void setBtnUse(boolean cat1, boolean cat2, boolean show,
                       boolean reset) {
    cat1Btn.setDisable(cat1);
    cat2Btn.setDisable(cat2);
    showBtn.setDisable(show);
    resetBtn.setDisable(reset);
}//end method setBtnUse
```

ORACLE Academy

Copyright © 2026, Oracle and/or its affiliates. Oracle®, Java, MySQL and NetSuite are registered trademarks of Oracle and/or its affiliates. Other names may be trademarks of their respective owners.

29. You can fix these issues by simply controlling access to the buttons.

    a. The reset button should be disabled until the `showVote` button is clicked. When the reset button is created set it to disabled:

```
resetBtn.setPrefWidth(BTN_WIDTH); resetBtn.setDisable(true);
```

    b. In the event handler for the `showBtn` call the `setBtnUse` method after the `showVotes()` call passing the following values:

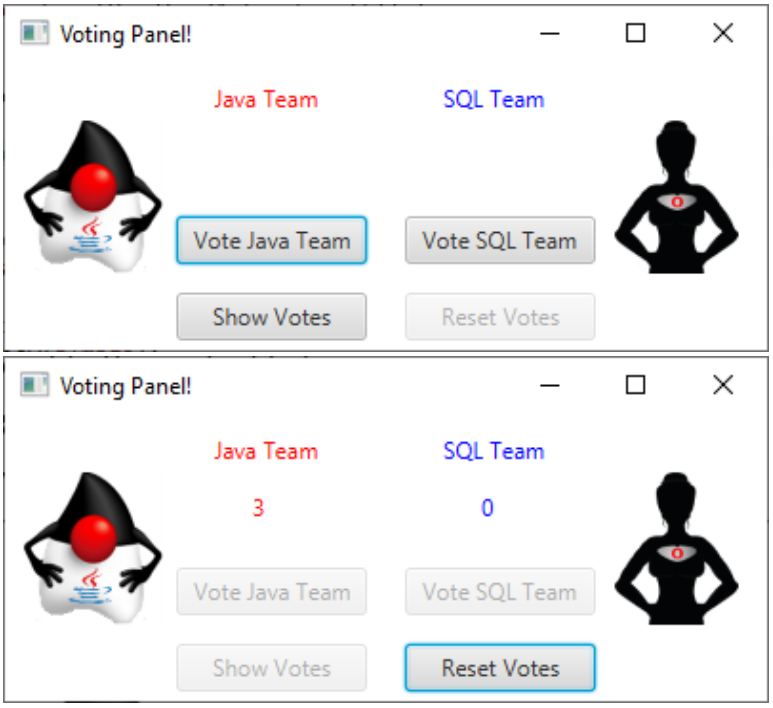
```
public void handle(ActionEvent event) {
    showVotes();
    setBtnUse(true, true, true, false);
}//end method handle
```

    This will disable the two vote buttons and the show button and enable the reset votes button.

    c. In the event handler for the `resetBtn` call the `setBtnUse` method passing values so that the two vote buttons and the show votes button are enabled, and the reset button is disabled.

    d. Run and test your program

    e. Your application should look like this:



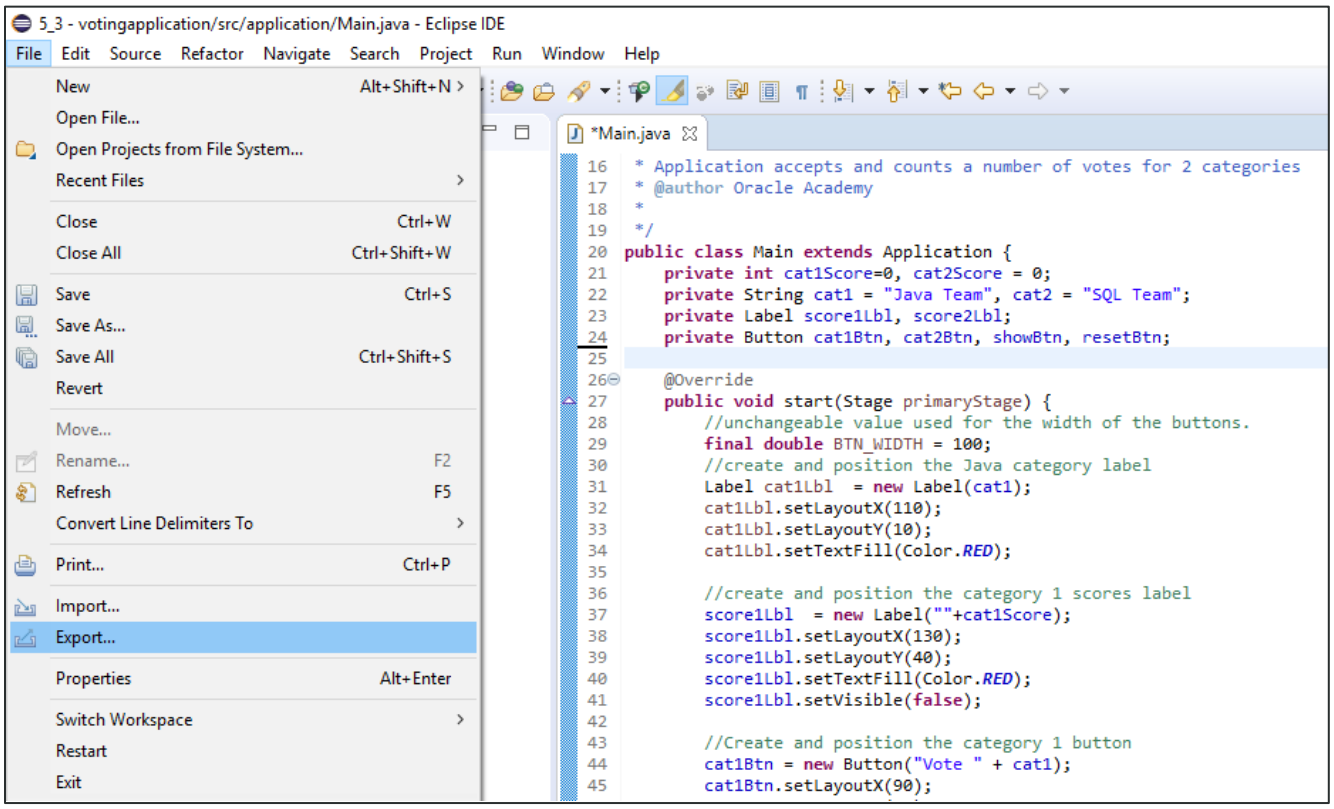
ORACLE Academy

Copyright © 2026, Oracle and/or its affiliates. Oracle®, Java, MySQL and NetSuite are registered trademarks of Oracle and/or its affiliates. Other names may be trademarks of their respective owners.

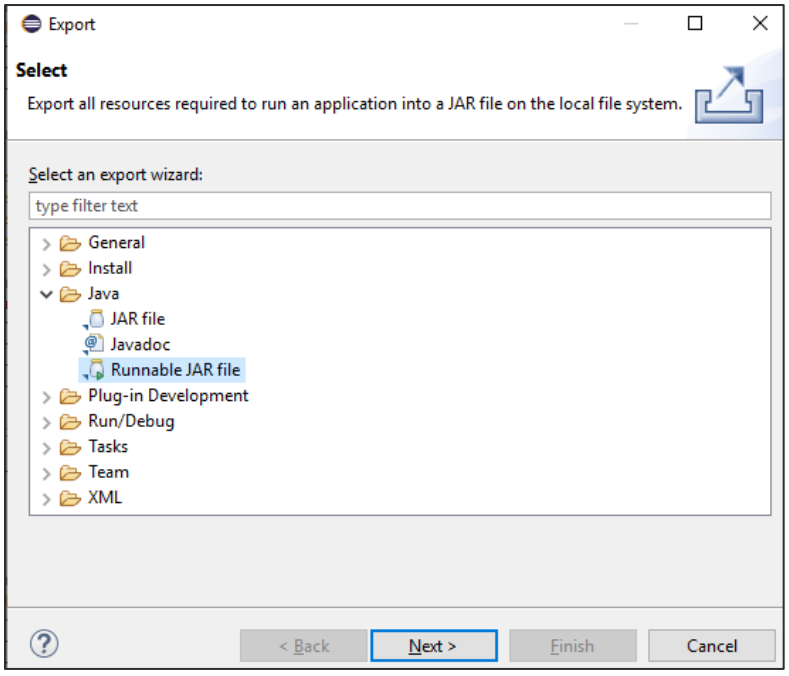
Creating a JAR file for your application

To be able to run the application outside of the Eclipse IDE you need to create a runnable JAR file.

1. In Eclipse select **File** > **Export**.



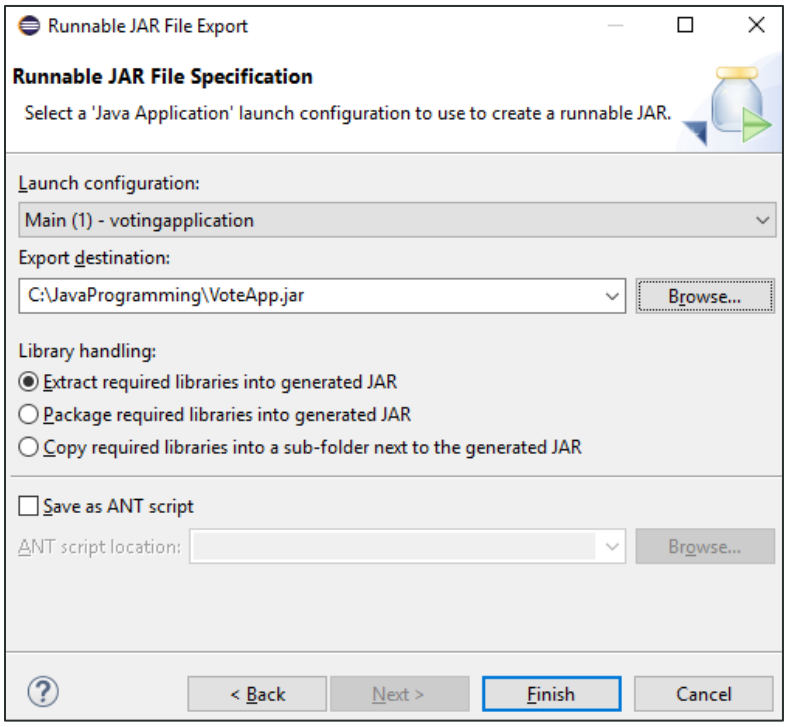
2. From the Export window, select **Java** > **Runnable JAR file** and click **Next >**.



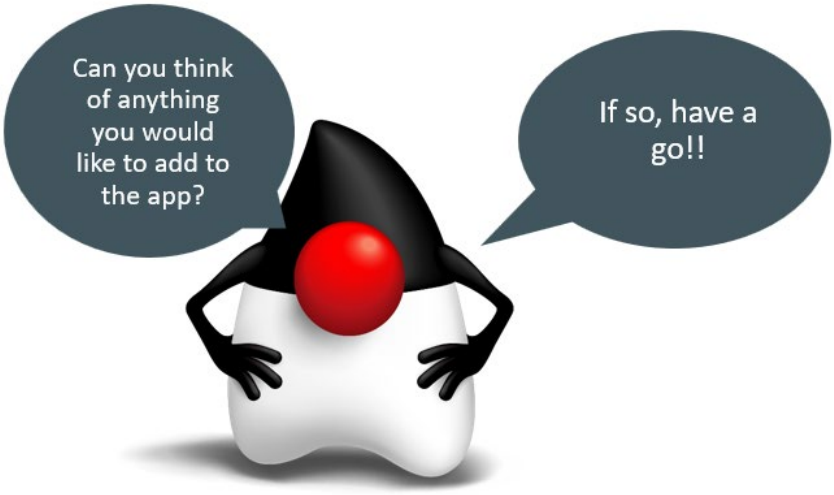
ORACLE Academy

Copyright © 2026, Oracle and/or its affiliates. Oracle®, Java, MySQL and NetSuite are registered trademarks of Oracle and/or its affiliates. Other names may be trademarks of their respective owners.

3. In the **Launch configuration** box, select the **Main** class from the application you are JARring up. Select the destination directory and name that you want to give your JAR file. Click **Finish** to generate the JAR file, click **OK** to any messages that appear.



4. Go to your chosen directory, double-click the JAR file to launch it!



ORACLE Academy

Copyright © 2026, Oracle and/or its affiliates. Oracle®, Java, MySQL and NetSuite are registered trademarks of Oracle and/or its affiliates. Other names may be trademarks of their respective owners.